

ChronoCampus

Developer Guide

Smart Time-Aware University Infrastructure Management System

CO2060 Software System Design Project
Department of Computer Engineering
Faculty of Engineering
University of Peradeniya

Document Information

Item	Details
Project	ChronoCampus
Course	CO2060 Software System Design Project
Document	Developer Guide
Backend	Python Flask
Frontend	HTML, CSS, JavaScript
Database	PostgreSQL
Version Control	Git and GitHub
Repository	https://github.com/cepdnack/e23-co2060-ChronoCampus
Source basis	Final project folder supplied for this guide

Purpose of This Guide

This guide explains how the ChronoCampus system is organized, configured, executed, maintained, and extended. It is based on the final project folder supplied with the request. The guide describes the implementation that is actually present in the repository rather than an idealized architecture.

1. System Overview

ChronoCampus is a web-based university infrastructure management system. The repository describes the system as a platform for academic schedule visibility, room and laboratory availability, facility reservations, staff locations, and notifications.

Area	Implemented Functionality
Authentication	Registration, login, logout, password change, role determination
Room visibility	Current room status, daily room schedule, room search, free-slot calculation
Reservations	Create, view, approve, reject, cancel reservations
Conflict handling	Booking conflict detection, alternative room suggestion, staff override request
Watchlist	Watch a room/time slot and receive notifications when it becomes available
Waitlist	Join a queue for an occupied room/time slot and receive queue notifications
Notifications	User notifications, read status, booking reminders

Area	Implemented Functionality
User directory	Search/display user and staff information and locations
Administration	User management and reservation-related administration pages

2. Technology Stack

Technology	Purpose
Python	Backend application development
Flask 3.1.2	Web framework and HTTP routing
Flask-SQLAlchemy	Object-relational database integration
psycopg2-binary	Direct PostgreSQL connections
PostgreSQL	Relational database
HTML	Page structure
CSS	User interface styling
JavaScript	Client-side logic and API communication
Flask-CORS	Cross-origin request support
bcrypt	Password hashing and verification
python-dotenv	Environment variable loading
Git / GitHub	Version control and collaboration

3. Repository Structure

```
e23-co2060-ChronoCampus/
├── README.md
├── code/
│   ├── backend/
│   │   ├── app.py
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── requirements.txt
│   │   ├── models/
│   │   └── routes/
│   ├── database/
│   │   └── schema.sql
│   ├── frontend/
│   │   ├── *.html
│   │   ├── *.css
│   │   └── js/
│   ├── images/
│   └── schema.sql
├── docs/
└── .gitignore
```

The supplied ZIP also contains local Python virtual-environment folders. These are development artifacts and are not required for a fresh installation when dependencies are installed from requirements.txt.

4. Backend Architecture

The Flask backend is centered on app.py. It creates the Flask application, configures CORS, initializes SQLAlchemy, registers the reservation, watchlist, and authentication blueprints, and defines room, directory, staff, administration, and page routes.

Component	Responsibility
app.py	Application creation, configuration, page routes, room APIs, directory/staff APIs and direct database operations
config.py	Reads database settings from environment variables and builds the SQLAlchemy connection URI
database.py	Creates the SQLAlchemy db object
models/	Database entity definitions and serialization methods
routes/auth_routes.py	Registration, login, logout and password-change APIs
routes/reservation_routes.py	Rooms, reservations, conflicts, approvals/cancellations, notifications, watchlist, waitlist and override handling
routes/watchlist_routes.py	Watchlist API endpoint
routes/schedule.py	Schedule CRUD blueprint present in the repository

5. Database Configuration

config.py loads database settings from environment variables. The expected variables are:

```
DB_HOST=localhost
DB_NAME=chronocampus_db
DB_USER=postgres
DB_PASSWORD=yourpassword
DB_PORT=5432
```

The application also defines SECRET_KEY in config.py and reads SECRET_KEY from the environment when creating the Flask application. Keep credentials and secrets outside source control.

6. Database Design

The supplied SQL schema defines the following main tables.

Table	Purpose
users	Registered users, roles, departments and account state
rooms	Lecture halls, laboratories and other rooms

Table	Purpose
reservations	Room reservation records and status
override_requests	Staff-to-student booking override requests
room_watchlist	Users watching a room/time slot
notifications	Messages delivered to users
canceled_reservations	Cancellation records and reasons
room_waitlist	Queue entries for occupied reservation slots
find_users	Directory information such as department, office location and profile image

The rooms table is seeded with sample rooms including Hall A, Hall B, Lecture Room 1, Lecture Room 2, Lab 1, Lab 2, Computer Lab A, Computer Lab B, and seminar rooms.

7. Database Setup

1. Install PostgreSQL and make sure the PostgreSQL service is running.
2. Create the database used by the application:

```
CREATE DATABASE chronocampus_db;
```

3. Connect to the database using psql or pgAdmin.
4. Run the project SQL schema. The repository contains code/database/schema.sql and code/schema.sql; use the schema version that your team has designated as the final deployment schema.
5. Create code/backend/.env with the database connection values.
6. Start Flask and confirm that the application can connect to PostgreSQL.

Important repository note: the supplied SQL files contain some duplicated/legacy statements and should be reviewed before using them as a clean production migration. In particular, the users table definition and later ALTER/INSERT statements are not fully consistent. This guide therefore does not claim that every raw SQL statement in the supplied file can be executed unchanged on a fresh database.

8. Backend Installation

```
cd code/backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

The repository README also lists the principal packages explicitly: Flask, flask-cors, flask-sqlalchemy, pycpg2-binary, bcrypt, and python-dotenv.

On Linux/macOS, activate the environment with:

```
source .venv/bin/activate
```

9. Running the Application

7. Activate the Python virtual environment.
8. Verify the PostgreSQL database is running.
9. Verify the .env database configuration.
10. Change to code/backend.
11. Run: `python app.py`
12. Open the Flask application at: `http://127.0.0.1:5000/`

The application configures its template and static folders to point to code/frontend, so the HTML/CSS/JavaScript files are served by Flask.

10. Frontend Architecture

The frontend is a collection of HTML pages supported by shared CSS and JavaScript modules. The major pages include login, signup, dashboard, room management, booking, reservations, directory, staff pages, and administrative pages.

Frontend Component	Purpose
login.html / signup.html	Authentication UI
dashboard.html	Main user dashboard
index2.html / room pages	Room visibility and availability functions
booking.html	Room reservation interface
my_reservations.html	User reservation list, filtering and cancellation
directory.html	User/staff directory
staff.html / staff_profile.html	Staff information and profiles
admin.html / admin4.html	Administrative interfaces
js/api.js	Centralized client-side API calls
js/auth.js	Authentication state and navigation helpers
js/booking.js	Booking page behavior
js/reservations.js	Reservation display/filter/cancel behavior
js/notifications.js	Notification loading and polling
js/datepicker.js	Date-picker related behavior

11. Room Availability Module

The room-management implementation in app.py provides four main APIs.

Endpoint	Method	Function
/rooms/current-status	GET	Returns every room as Free or Occupied based on the current time and today's reservations
/rooms/schedule?date=YYYY-MM-DD	GET	Returns reservations for each room on a selected date
/rooms/search?name=...&date=...	GET	Searches rooms by name and returns their bookings for the selected date
/rooms/availability?room_id=...&date=...	GET	Calculates free time slots between 08:00 and 17:00

Current status checks the rooms table and searches reservations whose reservation date is the current date and whose booking interval contains the current timestamp.

The availability calculation uses a working interval from 08:00 to 17:00. Reservations are retrieved in start-time order. A pointer begins at 08:00; whenever the next reservation begins after the pointer, the interval between the pointer and the reservation start is recorded as a free slot. The pointer is then moved to the later of its current value and the reservation end. Any remaining time before 17:00 is added as the final free slot.

12. Reservation Module

The reservation blueprint in routes/reservation_routes.py implements room retrieval, reservation creation, reservation listing, approval/rejection, cancellation, conflict handling, watchlists, waitlists, override requests, and notification-related operations.

Reservation creation validates:

- user_id and room_id are integers.
- Start and end timestamps are valid.
- End time is after start time.
- The requested start is at least three hours in advance.
- The requested start is not more than seven days in advance.
- The booking duration does not exceed six hours.
- The booking is not in the past.
- The user and room exist.
- The requested interval does not overlap an active reservation.

When a conflict exists, the system can provide an alternative room suggestion. It also exposes a queue option. If the requester is staff and the existing reservation belongs to a student, an override request is created and the student is notified.

13. Alternative Room Suggestion

The reservation module searches for other rooms whose capacity is at least the capacity of the originally selected room. Candidates are ordered by capacity. A candidate is suggested when there is no conflicting active reservation for the requested interval.

14. Watchlist and Waitlist

The watchlist allows a user to monitor a room/time slot. The waitlist provides an ordered queue for an occupied room/time slot.

Feature	Implementation
Watchlist	POST /watchlist, GET /watchlist/user/<user_id>, DELETE /watchlist/<watch_id>
Waitlist	POST /waitlist/join, GET /waitlist/user/<user_id>, DELETE /waitlist/<waitlist_id>
Queue position	First user receives position 1; later users receive increasing positions
Queue notification	When a slot opens, the first waiting user is notified and later positions are shifted

15. Authentication

Authentication is implemented in routes/auth_routes.py using PostgreSQL through psycopg2 and bcrypt for password hashing.

Endpoint	Method	Purpose
/auth/register	POST	Validate registration data, determine role and create a user
/auth/login	POST	Validate credentials and return user information
/auth/logout	POST	Return logout confirmation
/auth/change-password	PUT	Validate the old password and update the password

Registration restricts departments to the ALLOWED_DEPARTMENTS list and requires a university email pattern. Student and staff roles are inferred from the email format, while the configured administrator email is assigned the admin role.

The frontend stores selected user information in sessionStorage after login. The supplied authentication code should therefore be treated as the current project implementation rather than a claim of a complete server-side authorization framework.

16. Notifications

The reservation blueprint provides notification retrieval, read-state updates and reminder generation. Notifications are stored in the notifications table.

- Watchlist availability can create a notification.
- Waitlist position changes can create notifications.
- A staff override request creates a notification for the affected student.
- Approved reservations approaching their start time can generate reminders.

The frontend notification module polls/loads notifications and displays them through the notification panel.

17. User Directory and Staff Information

app.py provides a directory API that combines user records with directory-specific find_users information. It can expose department, office location, profile image and email information. Staff profile routes provide a staff page and a staff API endpoint.

Images are served from the project's image/static locations through Flask routes.

18. API Communication

The frontend communicates with Flask using the browser Fetch API. The shared api.js module provides API helper functions for application features, while feature-specific JavaScript files consume those helpers or call endpoints directly.

```
Frontend
  |
  | HTTP GET / POST / PUT / DELETE
  v
Flask routes / blueprints
  |
  v
SQLAlchemy models or psycopg2
  |
  v
PostgreSQL
  |
  v
JSON response
  |
  v
JavaScript updates the HTML UI
```

19. Important API Reference

Method	Endpoint	Purpose
GET	/rooms	List rooms
GET	/rooms/current-status	Current Free/Occupied status
GET	/rooms/schedule	Room bookings by date
GET	/rooms/search	Search room and view bookings
GET	/rooms/availability	Calculate free slots
POST	/reservations	Create reservation
GET	/reservations	List reservations
GET	/reservations/user/<user_id>	List a user's reservations
POST	/reservations/<reservation_id>/cancel	Cancel reservation
POST	/reservations/<reservation_id>/approve	Approve reservation
POST	/reservations/<reservation_id>/reject	Reject reservation
POST	/watchlist	Add watchlist entry
GET	/watchlist/user/<user_id>	Get user's watchlist
DELETE	/watchlist/<watch_id>	Remove watchlist entry
POST	/waitlist/join	Join waitlist
GET	/waitlist/user/<user_id>	Get user's waitlist
DELETE	/waitlist/<waitlist_id>	Leave waitlist
GET	/notifications/user/<user_id>	Get notifications
POST	/notifications/<notification_id>/read	Mark notification as read

20. Git and Collaboration Workflow

The repository README defines a three-level workflow:

- main: stable, demo-ready code.
- dev: integration branch where features are merged and tested.
- feature branches: individual module development branches merged into dev.

The repository history supplied with the final folder also contains module branches associated with authentication, schedule management, facility/reservation work, and notification/staff work.

```
feature branch
  |
  | pull request / merge
  v
dev
  |
  | tested integration
  v
main
```

Before starting work, developers should pull the latest branch state. Changes should be committed in small logical units. A feature should be integrated into dev and tested before dev is promoted to main.

21. Testing and Verification

The final project folder contains application code and database scripts, but it does not provide a complete automated test suite or a formal test-coverage report. Therefore, this guide documents practical verification procedures rather than claiming an automated coverage percentage.

Test Area	Recommended Verification
Database	Create database, run schema, verify tables and foreign keys
Authentication	Register valid/invalid users, test duplicate email, login with correct/incorrect password
Room status	Create an active reservation and verify Occupied; verify a room without an active reservation is Free
Daily schedule	Query a date with and without reservations
Search	Search full and partial room names
Availability	Test no bookings, one booking, multiple bookings, booking at 08:00 and booking ending at 17:00
Reservations	Test valid booking, overlap, past booking, >6 hour booking, <3 hour advance booking, >7 day booking
Cancellation	Cancel pending/approved reservation and verify status
Watchlist	Add duplicate and unique watch entries
Waitlist	Join, view position, leave, and verify position shifting
Notifications	Verify generated notification and read-state update
Admin	Test user-management and reservation-management operations

22. Troubleshooting

Problem	Checks
ModuleNotFoundError	Activate the correct virtual environment and run <code>pip install -r requirements.txt</code>
PostgreSQL connection error	Check PostgreSQL service, DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD
Database table missing	Run the selected final schema and verify the active database
Frontend cannot reach Flask	Confirm Flask is running on 127.0.0.1:5000 and inspect browser console/network errors
CORS error	Confirm Flask-CORS is installed and the application is running with its configured CORS settings
Login fails	Check user record, password hash, role/email rules and database connection
Room availability looks wrong	Check reservation timestamps, reservation status, selected date and the 08:00-17:00 working interval

23. Maintenance and Extension Guidelines

- Keep database credentials and secret keys in environment variables.
- Do not commit .env files or local virtual environments.
- Use feature branches for new functionality.
- Keep database changes synchronized with SQLAlchemy models and schema scripts.
- Test reservation conflict logic whenever reservation rules are changed.
- When adding an API endpoint, document its method, URL, parameters, request body and response format.
- Keep frontend API calls synchronized with backend route names.
- Use database migrations or controlled schema updates for future production deployments rather than repeatedly dropping all tables.

24. Current Repository Implementation Notes

The following points are important for developers maintaining this exact supplied repository:

- `app.py` directly defines several room, directory, staff and page routes while reservation/authentication functionality is separated into blueprints.
- A schedule model and `routes/schedule.py` are present, but the current `app.py` shown in the final folder does not register the schedule blueprint. The room daily schedule endpoint in `app.py` is therefore a separate implemented feature.

- The repository contains both `code/database/schema.sql` and `code/schema.sql`. They should be treated as schema sources that require team agreement on which one is authoritative.
- The SQL schema contains duplicated/legacy statements, including later department alterations and seed inserts that do not consistently match the earlier users definition. Clean deployment should validate the schema on a fresh database before evaluation or reuse.
- The supplied repository contains local virtual-environment files. A new developer should create a fresh virtual environment instead of depending on the bundled environment.
- The final Git working tree in the supplied ZIP contains modified/uncommitted files. Before a formal release, the team should verify the intended final commit and ensure the repository is clean.

25. Developer Quick Start

13. Clone the repository
14. Install PostgreSQL
15. Create the ChronoCampus database
16. Apply the agreed final SQL schema
17. Create `code/backend/.env`
18. `cd code/backend`
19. `python -m venv .venv`
20. `.venv\Scripts\activate`
21. `pip install -r requirements.txt`
22. `python app.py`
23. Open `http://127.0.0.1:5000/`
24. Test login, rooms, booking and administration features

26. Final Developer Checklist

- PostgreSQL is running.
- The correct database exists.
- The final schema has been applied successfully.
- `.env` contains the correct local database settings and is not committed.
- The Python virtual environment is activated.
- `requirements.txt` has been installed.
- Flask starts without import errors.
- Room status and availability APIs return expected JSON.
- Authentication registration and login work.
- Reservation conflict rules work.

- Cancellation and approval functions work.
- Watchlist and waitlist functions work.
- Notifications are displayed.
- Admin pages load correctly.
- The intended final Git commit is on the correct branch.

Appendix A: Main Backend Files

File	Description
code/backend/app.py	Main Flask application and direct APIs/page routes
code/backend/config.py	Configuration and database URI
code/backend/database.py	SQLAlchemy database object
code/backend/models/users.py	User model
code/backend/models/room.py	Room model
code/backend/models/reservation.py	Reservation model
code/backend/models/notification.py	Notification model
code/backend/models/watchlist.py	Watchlist model
code/backend/models/waitlist.py	Waitlist model
code/backend/models/override_request.py	Override request model
code/backend/models/schedule.py	Schedule model
code/backend/routes/auth_routes.py	Authentication APIs
code/backend/routes/reservation_routes.py	Reservation and related APIs
code/backend/routes/watchlist_routes.py	Watchlist API
code/backend/routes/schedule.py	Schedule CRUD blueprint

Appendix B: Main Frontend Files

File	Description
code/frontend/login.html	Login interface
code/frontend/signup.html	Registration interface
code/frontend/dashboard.html	Dashboard
code/frontend/booking.html	Booking interface
code/frontend/my_reservations.html	Reservation management
code/frontend/directory.html	Directory interface
code/frontend/staff.html	Staff listing

File	Description
code/frontend/staff_profile.html	Staff profile
code/frontend/admin.html	Administration interface
code/frontend/js/api.js	API helper functions
code/frontend/js/auth.js	Authentication/navigation logic
code/frontend/js/booking.js	Booking logic
code/frontend/js/reservations.js	Reservation page logic
code/frontend/js/notifications.js	Notification logic

Appendix C: Key Room Availability Algorithm

```

working_start = selected_date + 08:00
working_end   = selected_date + 17:00
current_pointer = working_start

for each reservation ordered by start time:
    if reservation.start > current_pointer:
        add [current_pointer, reservation.start] as a free slot
        current_pointer = max(current_pointer, reservation.end)

if current_pointer < working_end:
    add [current_pointer, working_end] as a free slot

```

This algorithm derives free periods from reservations. Free periods do not need to be stored as separate reservation records.

Appendix D: Source Repository

ChronoCampus GitHub repository:

<https://github.com/cepdnack/e23-co2060-ChronoCampus>